Netflix Database

Listed In

- Show ID
- Type
- Title
- Cast
- Country
- Date Added
- Rating
- Duration
- Release Year
- Description

Counting the null values

```
SELECT
        COUNT(*) FILTER(WHERE show_id IS NULL) AS Id_null,
        COUNT(*) FILTER(WHERE type IS NULL) AS type_null,
        COUNT(*) FILTER(WHERE title IS NULL) AS title_null,
        COUNT(*) FILTER(WHERE director IS NULL) AS director_null,
        COUNT(*) FILTER(WHERE "cast" IS NULL) AS cast_null,
        COUNT(*) FILTER(WHERE country IS NULL) AS country_null,
        COUNT(*) FILTER(WHERE date_added IS NULL) AS date_added_null,
        COUNT(*) FILTER(WHERE release_year IS NULL) AS release_year_null,
        COUNT(*) FILTER(WHERE rating IS NULL) AS rating_null,
        COUNT(*) FILTER(WHERE duration IS NULL) AS duration_null,
        COUNT(*) FILTER(WHERE listed_in IS NULL) AS listed_in_null,
        COUNT(*) FILTER(WHERE description IS NULL) AS description_null
FROM mydata;
```

We can see that there are NULLS.

Director nulls	2634	**Date added nulls**	10
Cast nulls	825	**Rating nulls**	4
Country nulls	831	**Duration nulls**	3

Checking for duplicates

```
SELECT show_id, COUNT(*)
FROM mydata
GROUP BY show_id
HAVING COUNT(*) > 1
```

```
SELECT title, release_year, COUNT(*)
FROM mydata
GROUP BY title, release_year
HAVING COUNT(*) > 1
```

No duplicates were found

Populate Director column (CTE)

First split the cast column so we can normalize it by associating every actor with the movie or tv-show director

```
WITH cast_expanded AS (
        SELECT show_id, director, TRIM(actor) AS actor
        FROM mydata,
        UNNEST(STRING_TO_ARRAY("cast",',')) AS actor
        WHERE director IS NOT NULL
)
SELECT *
FROM cast_expanded
LIMIT 10;
```

Then find the relationship between them by counting the number of collaborations between them

```
WITH cast_expanded AS (
        SELECT show_id, director, TRIM(actor) AS actor
        FROM mydata,
        UNNEST(STRING_TO_ARRAY("cast",',')) AS actor
        WHERE director IS NOT NULL
)
SELECT
        TRIM(actor) AS actor,
        director,
        COUNT(*) AS collaborations
FROM cast_expanded
GROUP BY actor, director
HAVING COUNT(*) >= 3
ORDER BY collaborations DESC
```

Populate Director column (CTE)

```
WITH actor_director_pairs AS (
        SELECT
                TRIM(actor) AS actor,
                director,
                COUNT(*) AS collaborations
        FROM mydata,
        UNNEST(STRING_TO_ARRAY("cast", ',')) AS actor
        WHERE director IS NOT NULL
        GROUP BY actor, director
        HAVING COUNT(*) >=3),

    missing_director_titles AS(
            SELECT
                    m.show_id,
                    m.title,
                    TRIM(actor) AS actor
            FROM mydata m,
            UNNEST(STRING_TO_ARRAY("cast", ',')) AS actor
            WHERE director IS NULL)
```

```
SELECT
        md.show_id,
        md.title,
        ad.director,
        ad.collaborations
FROM missing_director_titles md
JOIN actor_director_pairs ad
ON md.actor = ad.actor
ORDER BY collaborations DESC
```

484 of the null value for directors can be populated because this query retrieve the missing titles director
and suggest the director it should have based on the collaboration of the title cast with the suggested director
giving a fair chance that this title "MOVIE/TV-SHOW" was directed by that director

PRESERVE THE DATA IN A NEW COLUMN AND THEN ADD IT TO ORIGINAL COLUMN

```
ALTER TABLE mydata
ADD COLUMN pop_director TEXT;
```

1

```
UPDATE mydata
SET director = pop_director
WHERE pop_director IS NOT NULL

UPDATE mydata
SET director = 'Not Given'
WHERE director IS NULL

ALTER TABLE mydata
DROP COLUMN pop_director
```

3

```
WITH actor_director_pairs AS (
        SELECT
                TRIM(actor) AS actor,
                director,
                COUNT(*) AS collaborations
        FROM mydata,
        UNNEST(STRING_TO_ARRAY("cast",      ','))
AS actor
        WHERE director IS NOT NULL
        GROUP BY actor, director
        HAVING COUNT(*) >=3
), missing_director_titles AS(
        SELECT
                m.show_id,
                m.title,
                TRIM(actor) AS actor
        FROM mydata m,
        UNNEST(STRING_TO_ARRAY("cast",      ','))
AS actor
        WHERE director IS NULL
)
UPDATE mydata m
SET pop_director = s.director
FROM (
        SELECT
                md.show_id,
                ad.director
        FROM missing_director_titles md
        JOIN actor_director_pairs ad
        ON md.actor = ad.actor ) s
WHERE m.show_id = s.show_id
```

2

Populate The Country Column Based On Director Column

```
SELECT COALESCE(n1.country, n2.country)
FROM mydata n1
JOIN mydata n2
ON n1.director = n2.director
AND n1.show_id <> n2.show_id
WHERE n1.country IS NULL;

UPDATE mydata
SET country = n2.country
FROM mydata n2
WHERE mydata.director = n2.director
AND mydata.show_id <> n2.show_id
AND mydata.country IS NULL;
```

Check If There Is Still Some Countries That Are Not Populated

```
SELECT
        director,
        country,
        date_added
FROM mydata
WHERE country IS NULL
```

Populate The Rest With Not Given

```
UPDATE mydata
SET country = 'Not Given'
WHERE country IS NULL
```

Populate The Null Values In Cast With Not Given Also

```
UPDATE mydata
SET "cast" = 'Not Given'
WHERE "cast" IS NULL
```

Treating Inconsistencies

```
UPDATE mydata
SET title = TRIM(title),
        director = TRIM(director),
        country = TRIM(country);
```

```
UPDATE mydata
SET rating = TRIM(rating),
        description = TRIM(description),
        "cast" = TRIM("cast"),
        listed_in = TRIM(listed_in);
```

Capitalize every letter of every word

```
UPDATE mydata
SET director = INITCAP(director),
        "cast" = INITCAP("cast"),
        country = INITCAP(country),
        listed_in = INITCAP(listed_in);
```

Normalize Values

```
SELECT DISTINCT rating, COUNT(*)
FROM mydata
GROUP BY DISTINCT rating
ORDER BY COUNT(*) ASC
```

FIRST THREE VALUES ARE NOT RATING BUT DURATIONS BELONG TO THE DURATION COLUMN.

Validating That Duration Values For These Values Are Null Using REGEX

```
SELECT rating, duration
FROM mydata
where rating ~ '^[0-9]+ min$’;
```

Transferring The Values To It’s Original Column

```
UPDATE mydata
SET duration = rating, rating = NULL
WHERE rating ~ '^[0-9]+ min$';
```

Deleting 7 Rows Of Rating Nulls 7 Of 8,000 Will Not Affect The Analysis

```
SELECT show_id, rating
FROM mydata
WHERE rating IS NULL

DELETE FROM mydata
WHERE show_id IN('s7313', 's6828',
's7538', 's5814', 's5795', 's5542', 's5990')
```

NOW THE RATING COLUMN WILL HAVE 7 NULL VALUES INSTEAD OF 4 DUE TO THE TRANSFER 3 OF IT’S VALUES TO DURATION COLUMN

ADJUSTING & CONVERTING DATA TYPES

```
ALTER TABLE mydata
ALTER COLUMN release_year TYPE INT
USING release_year::INT;

SELECT date_added
FROM mydata
WHERE date_added IS NOT NULL

ALTER TABLE mydata
ALTER COLUMN date_added TYPE DATE
USING TO_DATE(date_added, 'MONTH DD, YYYY')
```

```
ALTER TABLE mydata
ALTER COLUMN show_id TYPE VARCHAR(55)

ALTER TABLE mydata
ADD CONSTRAINT pk_show PRIMARY KEY (show_id);

ALTER TABLE mydata
ALTER COLUMN title SET NOT NULL;
```